

CMPE260 Interpreter Project Report

Azra Zülal Muhcu - 2024400372

Mustafa Şahin - 2023400162

1. Scoping Decision

This project was implemented using static scoping. We were not sure of which scope to implement, but an advantage of working with the static scope is that the program becomes easier to trace since the variable reference can be determined by looking at the program. It is ensured that the function uses variables that are visible only when the function is created. Even though we did not need to implement a stack system to remember the function calls for the dynamic scope, we had to additionally implement the closure and environment system.

```
let x = 10;

let f = fun() ->
    x
end;

let g = fun() ->
    let x = 20;
    f()
end;

print(g());
```

In the above example, if it was static scoping, the output would be 10. It captures the environment where $x = 10$. However, in dynamic scoping, since the caller function would be through the g to f , it would still use the x that is 20. Therefore, the output will be 20.

```
let x = 1;

let makeAdder = fun(y) ->
    fun(z) ->
        x + y + z
    end
end;

let add5 = makeAdder(5);

x = 100;

print(add5(3));
```

In this example, the static scope would print a 108. However, in dynamic scoping, the program would not be able to resolve what the value of `y` is because `makeAdder(5)` already returned.

2. Evaluation Strategy

Call-by-value evaluation means that all function arguments are evaluated before the function is called, and the resulting values are passed to the function. For example:

```
let ignore = fun(x) ->
  42
end;

ignore(1 / 0)
```

Under call-by-value, `1/0` is evaluated before the call, causing a division-by-zero error. Under call-by-name, the expression would not be evaluated because `x` is never used, so the function would simply return 42.

3. Closure Representation

Closures are represented by the Closure class. A closure contains three pieces of information: the function's parameter list, the function body, and the environment in which the function was defined. Storing the defining environment allows functions to access variables from surrounding scopes even after those scopes have finished executing.

4. Environment Management

The environment is implemented as a chain of frames. Each frame contains a dictionary of variable bindings and a pointer to its parent environment. When a function is called, a new environment frame is created and linked to the environment stored in the function's closure. Variable lookup searches the current frame first and then follows parent pointers until the variable is found or an error is raised. This structure implements lexical scoping and allows closures to access variables from outer scopes.

a. Required Tracing Exercise

```
let x = 1;
let f = fun(a) ->
  let y = a + x;
  fun () ->
    x = x + 1;
    y+x
  end
end;
let g = f(10);
print(g());
print(g());
```

1) **let x = 1;**

Outer closure (O)

x = 1

parent = none

2) **let f = fun(a) -> ...**

Now, a closure is created which we store in O:

Outer closure (O)

x = 1

f = <closure_f>

parent = none

3) **let g = f(10);**

f(10) works to create a new frame called E

Inner frame (E)

a = 10

parent -> O

4) **let y = a + x;**

Lookup of a: (1) first check its own frame to find a. Found in E, so we can set a = 10.

(2) then, check its own frame to find x: not in E, but found in O.

(3) thus, $y = 10 + 1 = 11$

(4) store in E:

Inner frame (E)

a = 10

y = 11

parent -> O

5) **fun () ->**

x = x + 1;

y+x

end

This creates another closure, called closure_g that remembers the environment in which it was created (E).

closure_g

parameters = none

```
body =  
x = x + 1;  
y + x  
parent -> E
```

Since the inner function is the last expression of f,
f(10) returns closure_g.

Therefore:

```
let g = f(10);
```

stores closure_g in O.

```
Outer frame (O)  
x = 1  
f = <closure_f>  
g = <closure_g>  
parent = none
```

```
6) print(g());
```

Calling g() creates a new frame E1.

```
Frame E1  
parent -> E
```

Current chain:

```
E1 -> E -> O
```

It needs to execute $x = x + 1$, therefore check E1 (not found), E (not found), and O ($x=1$).
 $x + 1 = 2$ so,

```
Outer frame (O)  
x = 2  
f = <closure_f>  
g = <closure_g>
```

Next, evaluate $y + x$: lookup y (not found in E1, found $y = 11$ in E). Lookup x (not found in E1 or E, found in O as $x = 2$). Compute $11 + 2 = 13$.

Therefore our first output will be 13.

```
7) print(g());
```

Calling `g()` again creates another frame `E2`.

`E2`

parent -> `E`

Execute `x = x+1` (lookup `x` in `E2`, `E`, and `O`). compute `2 + 1 = 3`. Therefore,

Outer frame (`O`)

`x = 3`

`f = <closure_f>`

`g = <closure_g>`

Now evaluate `y + x`. `E2` → `E` (found `y = 11`). `E2` → `E` → `O` (found `x = 3`). Compute `11 + 3 = 14`. Therefore the output is 14.

The frame that is modified by `x = x + 1` is `O` because that is where `x` is bound. The reason why `y` does not change between the two calls to `g()` is due to `y` being stored in frame `E` as 11. No assignment modifies `y` and every call to `g()` sees the same `y = 11`.

5. Challenges

Handling variable assignment across nested scopes was difficult. An assignment should update the nearest existing binding rather than always creating a new variable. This was resolved by implementing an `update()` method that searches through the environment chain and modifies the frame where the variable is found.

Building the parser required careful handling of operator precedence and function calls. Expressions such as arithmetic operations, comparisons, logical operators, and nested function calls needed to be parsed in the correct order. This was resolved by organizing the parser into multiple precedence levels so that expressions are interpreted correctly.

6. Grammar Ambiguity

In order to avoid ambiguities in grammar, we utilized a hierarchical system starting from the nonterminal `<expr>`. Once an `<expr>` is detected, the program first replaces it with the nonterminal `<or_expr>` which has the least precedence and thus is closer to the top of the parser. The `<or_expr>` might be replaced with one of two things: an actual expression including "or" or another expression that is higher up in the precedence order such as `<and_expr>` or a comparison expression.